Author: Gabriel Vance

Class: DSC680 - Applied Data Science

Professor: Amirfarrokh Iranitalab

Date: February 15, 2025

Project 2 Milestone 3: Stock Market Performance vs Elections

This notebook follows the problem statement in `Vance Project 2 Milestone 3.pdf` and answers:

1. Are future S&P 500 returns statistically similar under both major political parties?
2. Are incumbent political parties more likely to lose elections during periods of S&P 500 underperformance?
3. Given the current political landscape, what is the predicted stock market return from January 2026 to January 2027?

The workflow caches downloaded market data to local CSV files so data is not re-downloaded each run.

Approach

- Use `yfinance` for historical S&P 500 data (`^GSPC`) and cache it to `data/sp500_daily.csv` .
- Compute January-to-January returns as the annual performance measure.
- Add U.S. presidential party labels by year and election outcomes for presidential election years.
- Apply statistical tests, visual analysis, and a simple regression forecast model.

Note: This is a historical association analysis, not causal proof or trading advice.

In [1]:
```
from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import statsmodels.api as sm

from scipy import stats
from scipy.stats import fisher_exact
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.model_selection import TimeSeriesSplit

warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)
plt.rcParams["axes.titlesize"] = 14
plt.rcParams["axes.labelsize"] = 11

DATA_DIR = Path("data")
DATA_DIR.mkdir(parents=True, exist_ok=True)

pd.set_option("display.max_columns", 50)

print(f"Data directory: {DATA_DIR.resolve()}")
```

```
Data directory: /Users/gvance67/GitHub/data_science_classes_windows/winter20
25/dsc680_applied/project2/data
```

In [ ]:

```
def load_sp500(cache_path=DATA_DIR / "sp500_daily.csv", start="1950-01-01"):
    """Load cached S&P 500 daily data or download and cache it."""
    if cache_path.exists():
        df = pd.read_csv(cache_path, parse_dates=["Date"], index_col="Date")
        source = "cache"
    else:
        df = yf.download("^GSPC", start=start, progress=False, auto_adjust=T
        if isinstance(df.columns, pd.MultiIndex):
            df.columns = [c[0] for c in df.columns]
        keep_cols = [
            c for c in ["Open", "High", "Low", "Close", "Volume"] if c in df
        ]
        df = df[keep_cols].dropna(subset=["Close"])
        df.to_csv(cache_path, index_label="Date")
        source = "download"

    if "Close" not in df.columns:
        raise ValueError("Expected a Close column in S&P 500 dataset.")

    return df.sort_index(), source


sp500_daily, source = load_sp500()
print(f"Loaded S&P 500 daily rows: {len(sp500_daily):,} (source: {source})")
print(
    f"Date range: {sp500_daily.index.min().date()} to {sp500_daily.index.max
)
sp500_daily.head()
```

```
Loaded S&P 500 daily rows: 19,146 (source: cache)
Date range: 1950-01-03 to 2026-02-06
```

Out[ ]:

Date	Open	High	Low	Close	Volume
1950-01-03	16.66	16.66	16.66	16.66	1260000
1950-01-04	16.85	16.85	16.85	16.85	1890000
1950-01-05	16.93	16.93	16.93	16.93	2550000
1950-01-06	16.98	16.98	16.98	16.98	2010000
1950-01-09	17.08	17.08	17.08	17.08	2520000

In [ ]:

```
# Build January-to-January annual return series
jan_prices = sp500_daily[sp500_daily.index.month == 1].copy()
jan_prices["year"] = jan_prices.index.year
jan_first_close = jan_prices.groupby("year")["Close"].first()

annual_returns = (
    (jan_first_close.shift(-1) / jan_first_close - 1)
    .dropna()
    .to_frame("return_jan_to_jan")
)
annual_returns.index.name = "year"
annual_returns["return_pct"] = annual_returns["return_jan_to_jan"] * 100
annual_returns["is_presidential_election_year"] = (
    annual_returns.index % 4 == 0
).astype(int)
annual_returns["is_midterm_year"] = (annual_returns.index % 4 == 2).astype(i
annual_returns["is_even_year"] = (annual_returns.index % 2 == 0).astype(int)

# Presidential party mapping by year (party in power for most of the Jan-to-
PARTY_PERIODS = [
    (1950, 1952, "Democrat"),
    (1953, 1960, "Republican"),
    (1961, 1968, "Democrat"),
    (1969, 1976, "Republican"),
    (1977, 1980, "Democrat"),
    (1981, 1992, "Republican"),
    (1993, 2000, "Democrat"),
    (2001, 2008, "Republican"),
    (2009, 2016, "Democrat"),
    (2017, 2020, "Republican"),
    (2021, 2024, "Democrat"),
    (2025, 2026, "Republican"),
]


def get_party(year):
    for start, end, party in PARTY_PERIODS:
        if start <= year <= end:
            return party
    return np.nan
```

```
annual_returns["president_party"] = annual_returns.index.map(get_party)

annual_returns.to_csv(DATA_DIR / "sp500_annual_returns_jan_to_jan.csv", inde
annual_returns.to_csv(DATA_DIR / "annual_returns_with_politics.csv", index=T

annual_returns.head(10)
```

Out[ ]:

year	return_jan_to_jan	return_pct	is_presidential_election_year	is_midterm_year	i
1950	0.246699	24.669872	0	1	
1951	0.145883	14.588342	0	0	
1952	0.115126	11.512612	1	0	
1953	-0.059910	-5.990957	0	0	
1954	0.472946	47.294585	0	1	
1955	0.228844	22.884353	0	0	
1956	0.023029	2.302925	1	0	
1957	-0.127056	-12.705625	0	0	
1958	0.374659	37.465897	0	1	
1959	0.080628	8.062773	0	0	

Election-Cycle Return Comparison (Non-election vs Midterm vs Presidential years)

This section explicitly compares Jan-to-Jan S&P 500 returns across election-cycle year types.

In [ ]:

```
cycle_df = annual_returns.reset_index().copy()
cycle_df["cycle_type"] = np.select(
    [cycle_df["year"] % 4 == 0, cycle_df["year"] % 4 == 2],
    ["Presidential election year", "Midterm year"],
    default="Non-election year",
)
order = ["Non-election year", "Midterm year", "Presidential election year"]

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.boxplot(
    data=cycle_df,
    x="cycle_type",
    y="return_pct",
    order=order,
    palette=["#4c78a8", "#f58518", "#54a24b"],
    ax=axes[0],
)
axes[0].axhline(0, color="black", linestyle="--", linewidth=1)
axes[0].set_title("Distribution of Returns by Election-Cycle Year Type")
```

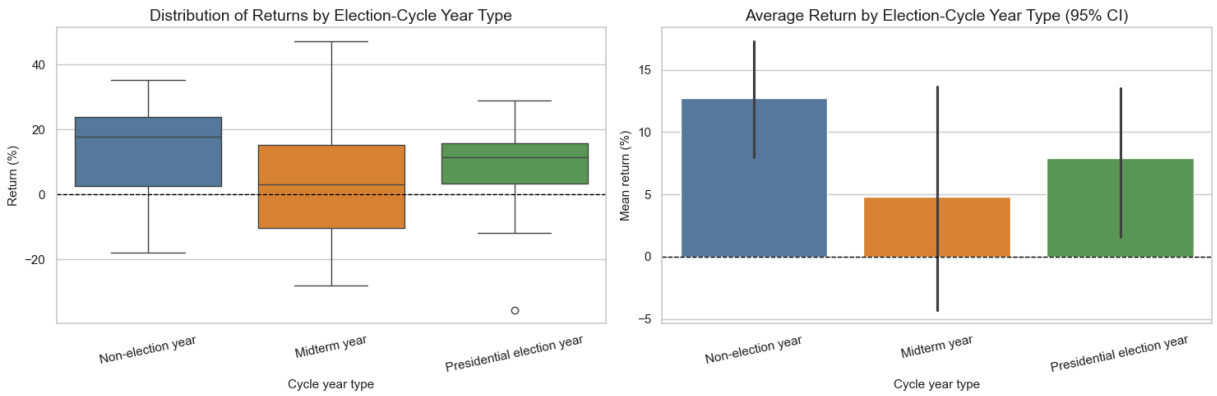
```
axes[0].set_xlabel("Cycle year type")
axes[0].set_ylabel("Return (%)")
axes[0].tick_params(axis="x", rotation=12)

sns.barplot(
    data=cycle_df,
    x="cycle_type",
    y="return_pct",
    order=order,
    estimator=np.mean,
    errorbar=("ci", 95),
    palette=["#4c78a8", "#f58518", "#54a24b"],
    ax=axes[1],
)
axes[1].axhline(0, color="black", linestyle="--", linewidth=1)
axes[1].set_title("Average Return by Election-Cycle Year Type (95% CI)")
axes[1].set_xlabel("Cycle year type")
axes[1].set_ylabel("Mean return (%)")
axes[1].tick_params(axis="x", rotation=12)

plt.tight_layout()
plt.show()

grp = cycle_df.groupby("cycle_type")["return_jan_to_jan"]
summary = grp.agg(["count", "mean", "median", "std"]).reindex(order)
anova = stats.f_oneway(*(grp.get_group(g) for g in order))
kruskal = stats.kruskal(*(grp.get_group(g) for g in order))

summary_pct = summary.copy()
for c in ["mean", "median", "std"]:
    summary_pct[c] = summary_pct[c] * 100
summary_pct
```



Out[ ]:

cycle_type	count	mean	median	std
Non-election year	38	12.733054	17.680918	14.910224
Midterm year	19	4.813213	3.101688	20.402982
Presidential election year	19	7.893475	11.512612	14.199427

In [ ]:
```
from IPython.display import Markdown, display

cycle_means = (summary["mean"] * 100).to_dict()
text = f"""
### Election-cycle finding
Average Jan-to-Jan returns are:
- Non-election years: **{cycle_means['Non-election year']:.2f}%**
- Midterm years: **{cycle_means['Midterm year']:.2f}%**
- Presidential election years: **{cycle_means['Presidential election year']:

ANOVA p-value = **{anova.pvalue:.3f}**, Kruskal-Wallis p-value = **{kruskal.
Interpretation: differences may exist descriptively, but test significance d
"""
display(Markdown(text))
```

Election-cycle finding

Average Jan-to-Jan returns are:

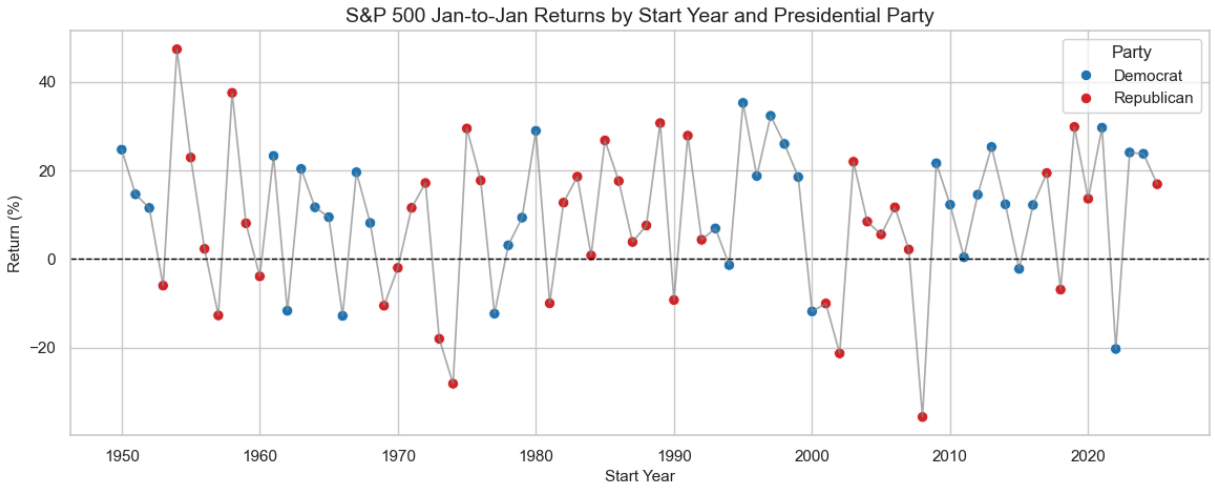
- Non-election years: **12.73%**
- Midterm years: **4.81%**
- Presidential election years: **7.89%**

ANOVA p-value = **0.203**, Kruskal-Wallis p-value = **0.172**. Interpretation: differences may exist descriptively, but test significance depends on variance and sample size.

Q1: Are future returns statistically similar under both major parties?

In [ ]:
```
palette = {"Democrat": "#1f77b4", "Republican": "#d62728"}
plot_df = annual_returns.reset_index().dropna(subset=["president_party"])

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(
    plot_df["year"], plot_df["return_pct"], color="#888888", linewidth=1.2,
)
sns.scatterplot(
    data=plot_df,
    x="year",
    y="return_pct",
    hue="president_party",
    palette=palette,
    s=55,
    ax=ax,
)
ax.axhline(0, color="black", linestyle="--", linewidth=1)
ax.set_title("S&P 500 Jan-to-Jan Returns by Start Year and Presidential Part
ax.set_xlabel("Start Year")
ax.set_ylabel("Return (%)")
ax.legend(title="Party")
plt.show()
```

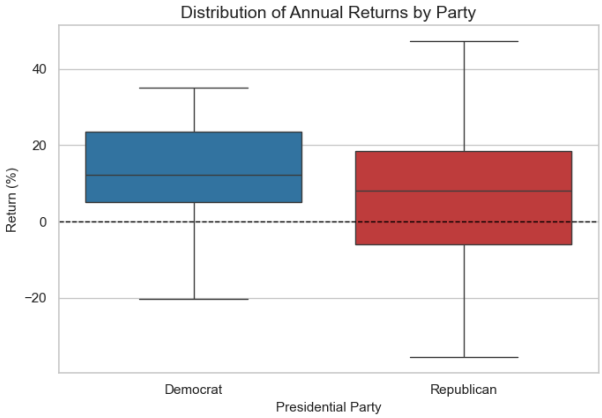


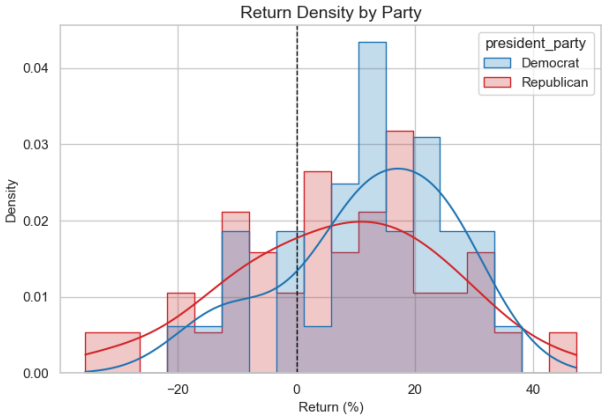
In [ ]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(
    data=plot_df,
    x="president_party",
    y="return_pct",
    order=["Democrat", "Republican"],
    palette=palette,
    ax=axes[0],
)
axes[0].axhline(0, color="black", linestyle="--", linewidth=1)
axes[0].set_title("Distribution of Annual Returns by Party")
axes[0].set_xlabel("Presidential Party")
axes[0].set_ylabel("Return (%)")

sns.histplot(
    data=plot_df,
    x="return_pct",
    hue="president_party",
    kde=True,
    stat="density",
    bins=18,
    palette=palette,
    ax=axes[1],
    element="step",
    common_norm=False,
)
axes[1].axvline(0, color="black", linestyle="--", linewidth=1)
axes[1].set_title("Return Density by Party")
axes[1].set_xlabel("Return (%)")
axes[1].set_ylabel("Density")

plt.tight_layout()
plt.show()
```





In [ ]:

```
dem = annual_returns.loc[
    annual_returns["president_party"] == "Democrat", "return_jan_to_jan"
].dropna()
rep = annual_returns.loc[
    annual_returns["president_party"] == "Republican", "return_jan_to_jan"
].dropna()

# Welch's t-test and Mann-Whitney U (robust non-parametric alternative)
test_t = stats.ttest_ind(dem, rep, equal_var=False, nan_policy="omit")
test_u = stats.mannwhitneyu(dem, rep, alternative="two-sided")

# Cohen's d effect size
pooled_sd = np.sqrt(((dem.var(ddof=1) + rep.var(ddof=1)) / 2))
cohens_d = (dem.mean() - rep.mean()) / pooled_sd

# Bootstrap confidence interval for mean difference (Dem - Rep)
rng = np.random.default_rng(680)
boot = []
for _ in range(5000):
    boot_dem = rng.choice(dem.to_numpy(), size=len(dem), replace=True).mean(
    boot_rep = rng.choice(rep.to_numpy(), size=len(rep), replace=True).mean(
    boot.append(boot_dem - boot_rep)
boot = np.array(boot)
ci_low, ci_high = np.percentile(boot, [2.5, 97.5])

q1_stats = pd.DataFrame(
    {
        "Metric": [
            "Democratic mean return",
            "Republican mean return",
            "Difference (Dem - Rep)",
            "Welch t-test p-value",
            "Mann-Whitney p-value",
            "Cohen's d",
            "Bootstrap 95% CI low (Dem-Rep)",
            "Bootstrap 95% CI high (Dem-Rep)",
        ],
        "Value": [
            dem.mean(),
            rep.mean(),
            dem.mean() - rep.mean(),
            test_t.pvalue,
```

```
            test_u.pvalue,
            cohens_d,
            ci_low,
            ci_high,
        ],
    }
)

q1_stats["Value"] = q1_stats["Value"].map(lambda x: float(x))
q1_stats
```

Out[ ]:

	Metric	Value
0	Democratic mean return	0.121602
1	Republican mean return	0.073092
2	Difference (Dem - Rep)	0.048511
3	Welch t-test p-value	0.192898
4	Mann-Whitney p-value	0.196276
5	Cohen's d	0.299613
6	Bootstrap 95% CI low (Dem-Rep)	-0.023807
7	Bootstrap 95% CI high (Dem-Rep)	0.119357

In [ ]:

```
from IPython.display import Markdown, display

q1_sig = (test_t.pvalue < 0.05) or (test_u.pvalue < 0.05)
q1_msg = (
    "not statistically distinguishable" if not q1_sig else "statistically di
)

text = f"""
### Critical answer for Question 1
The estimated average Jan-to-Jan return under Democratic presidencies is **{

However, the party-level return distributions are **{q1_msg}** using both a
The bootstrap 95% confidence interval for the mean difference (Dem - Rep) is

Interpretation: historical returns show overlap and large variance, so party
"""
display(Markdown(text))
```

Critical answer for Question 1

The estimated average Jan-to-Jan return under Democratic presidencies is **12.16%**, versus **7.31%** under Republican presidencies.

However, the party-level return distributions are **not statistically distinguishable** using both a Welch t-test (p = **0.193**) and Mann-Whitney test (p = **0.196**). The bootstrap 95% confidence interval for the mean difference (Dem - Rep) is **[-2.38%, 11.94%]**, which includes zero.

Interpretation: historical returns show overlap and large variance, so party label alone is a weak predictor of next-year S&P 500 performance.

Q2: Are incumbents more likely to lose during underperformance?

In [ ]:
```
election_cache = DATA_DIR / "us_presidential_elections_1952_2024.csv"

if election_cache.exists():
    elections = pd.read_csv(election_cache)
else:
    elections = pd.DataFrame(
        [
            {
                "election_year": 1952,
                "incumbent_party": "Democrat",
                "winner_party": "Republican",
            },
            {
                "election_year": 1956,
                "incumbent_party": "Republican",
                "winner_party": "Republican",
            },
            {
                "election_year": 1960,
                "incumbent_party": "Republican",
                "winner_party": "Democrat",
            },
            {
                "election_year": 1964,
                "incumbent_party": "Democrat",
                "winner_party": "Democrat",
            },
            {
                "election_year": 1968,
                "incumbent_party": "Democrat",
                "winner_party": "Republican",
            },
            {
                "election_year": 1972,
                "incumbent_party": "Republican",
```

```
        "winner_party": "Republican",
    },
    {
        "election_year": 1976,
        "incumbent_party": "Republican",
        "winner_party": "Democrat",
    },
    {
        "election_year": 1980,
        "incumbent_party": "Democrat",
        "winner_party": "Republican",
    },
    {
        "election_year": 1984,
        "incumbent_party": "Republican",
        "winner_party": "Republican",
    },
    {
        "election_year": 1988,
        "incumbent_party": "Republican",
        "winner_party": "Republican",
    },
    {
        "election_year": 1992,
        "incumbent_party": "Republican",
        "winner_party": "Democrat",
    },
    {
        "election_year": 1996,
        "incumbent_party": "Democrat",
        "winner_party": "Democrat",
    },
    {
        "election_year": 2000,
        "incumbent_party": "Democrat",
        "winner_party": "Republican",
    },
    {
        "election_year": 2004,
        "incumbent_party": "Republican",
        "winner_party": "Republican",
    },
    {
        "election_year": 2008,
        "incumbent_party": "Republican",
        "winner_party": "Democrat",
    },
    {
        "election_year": 2012,
        "incumbent_party": "Democrat",
        "winner_party": "Democrat",
    },
    {
        "election_year": 2016,
        "incumbent_party": "Democrat",
        "winner_party": "Republican",
```

```
            },
            {
                "election_year": 2020,
                "incumbent_party": "Republican",
                "winner_party": "Democrat",
            },
            {
                "election_year": 2024,
                "incumbent_party": "Democrat",
                "winner_party": "Republican",
            },
        ]
    )
    elections.to_csv(election_cache, index=False)

elections["incumbent_lost"] = (
    elections["incumbent_party"] != elections["winner_party"]
).astype(int)
ret = annual_returns["return_jan_to_jan"]


# Underperformance measured as compounded return over the two years before e
def get_pre2_return(election_year):
    y1, y2 = election_year - 2, election_year - 1
    if y1 in ret.index and y2 in ret.index:
        return (1 + ret.loc[y1]) * (1 + ret.loc[y2]) - 1
    return np.nan


elections["pre2y_return"] = elections["election_year"].map(get_pre2_return)
elections["pre2y_return_pct"] = elections["pre2y_return"] * 100
elections["underperformance"] = (elections["pre2y_return"] < 0).astype(int)

elections = elections.dropna(subset=["pre2y_return"]).copy()
elections.to_csv(DATA_DIR / "election_market_dataset.csv", index=False)

elections.head()
```

Out[ ]:

	election_year	incumbent_party	winner_party	incumbent_lost	pre2y_return	pre2y
0	1952	Democrat	Republican	1	0.428571	
1	1956	Republican	Republican	0	0.810020	
2	1960	Republican	Democrat	1	0.485495	
3	1964	Democrat	Democrat	0	0.062993	
4	1968	Democrat	Republican	1	0.042634	

In [ ]:

```
# Logistic trend: incumbent loss probability vs pre-election return
X_logit = sm.add_constant(elections[["pre2y_return"]], has_constant="add")
y_logit = elections["incumbent_lost"]
logit_model = sm.Logit(y_logit, X_logit).fit(disp=False)

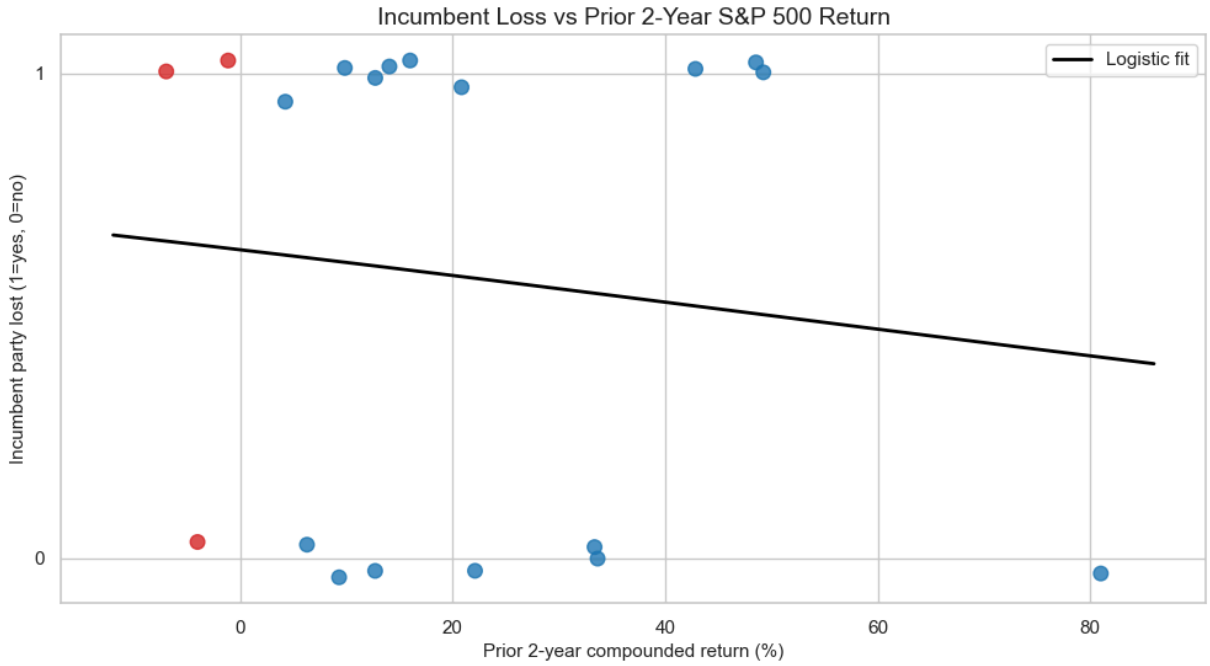
x_grid = np.linspace(
```

```
    elections["pre2y_return"].min() - 0.05, elections["pre2y_return"].max()
)
x_grid_df = pd.DataFrame({"pre2y_return": x_grid})
x_grid_sm = sm.add_constant(x_grid_df, has_constant="add")
p_grid = logit_model.predict(x_grid_sm)

fig, ax = plt.subplots(figsize=(12, 6))
rng = np.random.default_rng(42)
y_jitter = elections["incumbent_lost"] + rng.normal(0, 0.03, len(elections))

ax.scatter(
    elections["pre2y_return_pct"],
    y_jitter,
    c=elections["underperformance"].map({1: "#d62728", 0: "#1f77b4"}),
    s=70,
    alpha=0.8,
)
ax.plot(x_grid * 100, p_grid, color="black", linewidth=2, label="Logistic fi

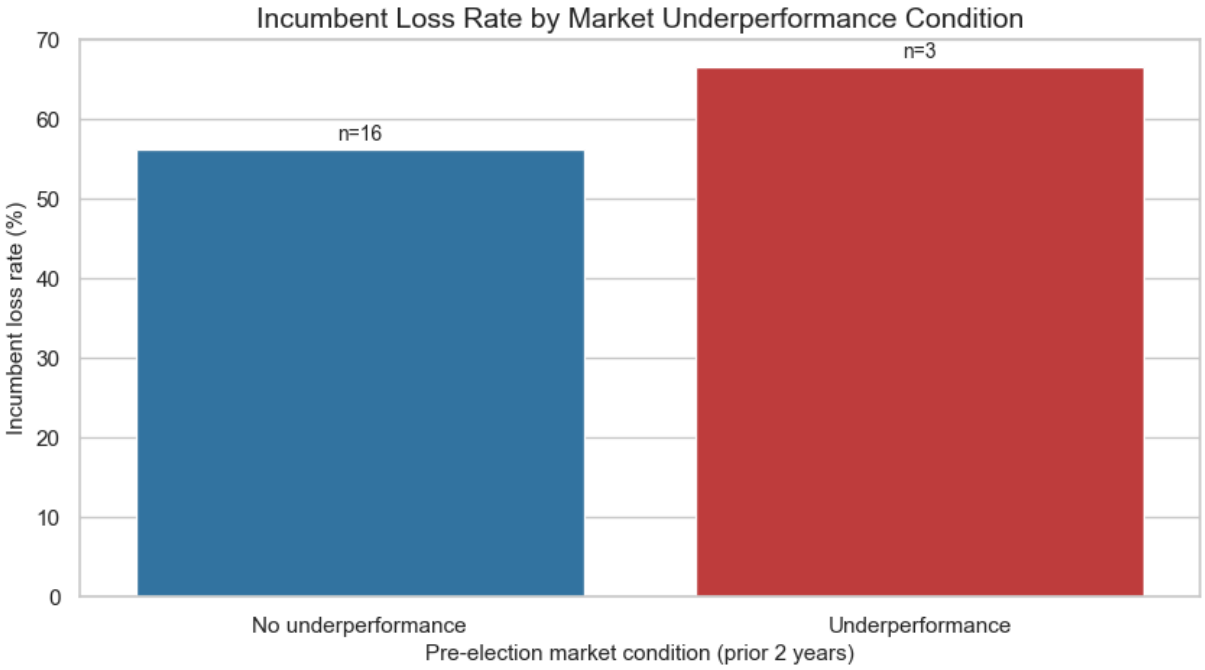
ax.set_title("Incumbent Loss vs Prior 2-Year S&P 500 Return")
ax.set_xlabel("Prior 2-year compounded return (%)")
ax.set_ylabel("Incumbent party lost (1=yes, 0=no)")
ax.set_yticks([0, 1])
ax.legend()
plt.show()
```



In [ ]:
```
loss_rate = (
    elections.groupby("underperformance")["incumbent_lost"]
    .mean()
    .rename(index={0: "No underperformance", 1: "Underperformance"})
)
counts = (
    elections.groupby("underperformance")["incumbent_lost"]
    .size()
    .rename(index={0: "No underperformance", 1: "Underperformance"})
```

```
)

fig, ax = plt.subplots(figsize=(10, 5))
sns.barplot(
    x=loss_rate.index, y=loss_rate.values * 100, palette=["#1f77b4", "#d6272
)
ax.set_title("Incumbent Loss Rate by Market Underperformance Condition")
ax.set_xlabel("Pre-election market condition (prior 2 years)")
ax.set_ylabel("Incumbent loss rate (%)")
for i, idx in enumerate(loss_rate.index):
    ax.text(
        i,
        loss_rate.loc[idx] * 100 + 1,
        f"n={counts.loc[idx]}",
        ha="center",
        fontsize=10,
    )
plt.show()
```



In [ ]:

```
contingency = pd.crosstab(elections["underperformance"], elections["incumber
odds_ratio, p_fisher = fisher_exact(contingency)
logit_p = float(logit_model.pvalues["pre2y_return"])

summary_q2 = pd.DataFrame(
    {
        "Metric": [
            "Loss rate during underperformance",
            "Loss rate during non-underperformance",
            "Fisher exact p-value",
            "Odds ratio (underperformance vs not)",
            "Logit slope p-value",
            "Logit slope coefficient",
        ],
        "Value": [
            loss_rate.loc["Underperformance"],
```

```
            loss_rate.loc["No underperformance"],
            p_fisher,
            odds_ratio,
            logit_p,
            float(logit_model.params["pre2y_return"]),
        ],
    }
)
summary_q2
```

Out[ ]:

	Metric	Value
0	Loss rate during underperformance	0.666667
1	Loss rate during non-underperformance	0.562500
2	Fisher exact p-value	1.000000
3	Odds ratio (underperformance vs not)	1.555556
4	Logit slope p-value	0.607566
5	Logit slope coefficient	-1.115766

In [ ]:

```
q2_sig = p_fisher < 0.05
q2_msg = (
    "suggests a meaningful relationship"
    if q2_sig
    else "does not establish a statistically significant relationship"
)

text = f"""
### Critical answer for Question 2
In this sample, incumbent parties lost **{loss_rate.loc['Underperformance']*
versus **{loss_rate.loc['No underperformance']*100:.1f}%** after non-underpe

But Fisher's exact test gives p = **{p_fisher:.3f}**, and the logistic slope
That means the data **{q2_msg}** at conventional significance thresholds, ma

Interpretation: there is directional evidence that weak markets hurt incumbe
"""
display(Markdown(text))
```

Critical answer for Question 2

In this sample, incumbent parties lost **66.7%** of elections after market underperformance periods, versus **56.2%** after non-underperformance periods.

But Fisher's exact test gives p = **1.000**, and the logistic slope p-value is **0.608**. That means the data **does not establish a statistically significant relationship** at conventional significance thresholds, mainly because the sample is small (presidential elections only).

Interpretation: there is directional evidence that weak markets hurt incumbents, but uncertainty is high and the relationship is not deterministic.

Q3: Predict Jan 2026 to Jan 2027 return

In [ ]:
```
model_df = annual_returns.copy()
model_df["party_republican"] = (model_df["president_party"] == "Republican")
model_df["lag1_return"] = model_df["return_jan_to_jan"].shift(1)
model_df["lag2_return"] = model_df["return_jan_to_jan"].shift(2)

feature_cols = [
    "party_republican",
    "is_presidential_election_year",
    "is_midterm_year",
    "lag1_return",
    "lag2_return",
]

model_df = model_df.dropna(subset=feature_cols + ["return_jan_to_jan"]).copy
X = model_df[feature_cols]
y = model_df["return_jan_to_jan"]

# Rolling-origin time series CV for realistic error estimate
cv = TimeSeriesSplit(n_splits=8)
cv_pred = pd.Series(index=model_df.index, dtype=float)
for train_idx, test_idx in cv.split(X):
    m = LinearRegression()
    m.fit(X.iloc[train_idx], y.iloc[train_idx])
    cv_pred.iloc[test_idx] = m.predict(X.iloc[test_idx])

mask = cv_pred.notna()
cv_mae = mean_absolute_error(y[mask], cv_pred[mask])
cv_rmse = np.sqrt(mean_squared_error(y[mask], cv_pred[mask]))
cv_r2 = 1 - np.sum((y[mask] - cv_pred[mask]) ** 2) / np.sum(
    (y[mask] - y[mask].mean()) ** 2
)
resid_std = (y[mask] - cv_pred[mask]).std(ddof=1)

# Fit final model on full training history
final_model = LinearRegression()
final_model.fit(X, y)
coef_series = pd.Series(final_model.coef_, index=feature_cols).sort_values()

model_df.to_csv(DATA_DIR / "model_training_dataset.csv", index=True)

pd.DataFrame(
    {
        "Metric": ["CV MAE", "CV RMSE", "CV R^2", "Residual Std (for PI)"],
        "Value": [cv_mae, cv_rmse, cv_r2, resid_std],
    }
)
```

Out[ ]:

	Metric	Value
0	CV MAE	0.163867
1	CV RMSE	0.247648
2	CV R^2	-1.398709
3	Residual Std (for PI)	0.246874

In [ ]:
```
# Build Jan 2026 -> Jan 2027 forecast input
forecast_year = 2026
required_years = [2024, 2025]
missing_years = [yr for yr in required_years if yr not in annual_returns.inc
if missing_years:
    raise ValueError(f"Missing required lag years for forecast: {missing_yea

forecast_input = pd.DataFrame(
    {
        "party_republican": [1],  # current landscape assumption for Jan 202
        "is_presidential_election_year": [1 if forecast_year % 4 == 0 else 0
        "is_midterm_year": [1 if forecast_year % 4 == 2 else 0],
        "lag1_return": [annual_returns.loc[2025, "return_jan_to_jan"]],
        "lag2_return": [annual_returns.loc[2024, "return_jan_to_jan"]],
    },
    index=[forecast_year],
)

forecast_return = float(final_model.predict(forecast_input)[0])
pi_low = forecast_return - 1.96 * resid_std
pi_high = forecast_return + 1.96 * resid_std

forecast_table = pd.DataFrame(
    {
        "Item": [
            "Predicted return (Jan 2026 to Jan 2027)",
            "95% prediction interval low",
            "95% prediction interval high",
        ],
        "Value (decimal)": [forecast_return, pi_low, pi_high],
        "Value (%)": [forecast_return * 100, pi_low * 100, pi_high * 100],
    }
)
forecast_table
```

Out[ ]:

	Item	Value (decimal)	Value (%)
0	Predicted return (Jan 2026 to Jan 2027)	-0.032792	-3.279249
1	95% prediction interval low	-0.516665	-51.666544
2	95% prediction interval high	0.451080	45.108047

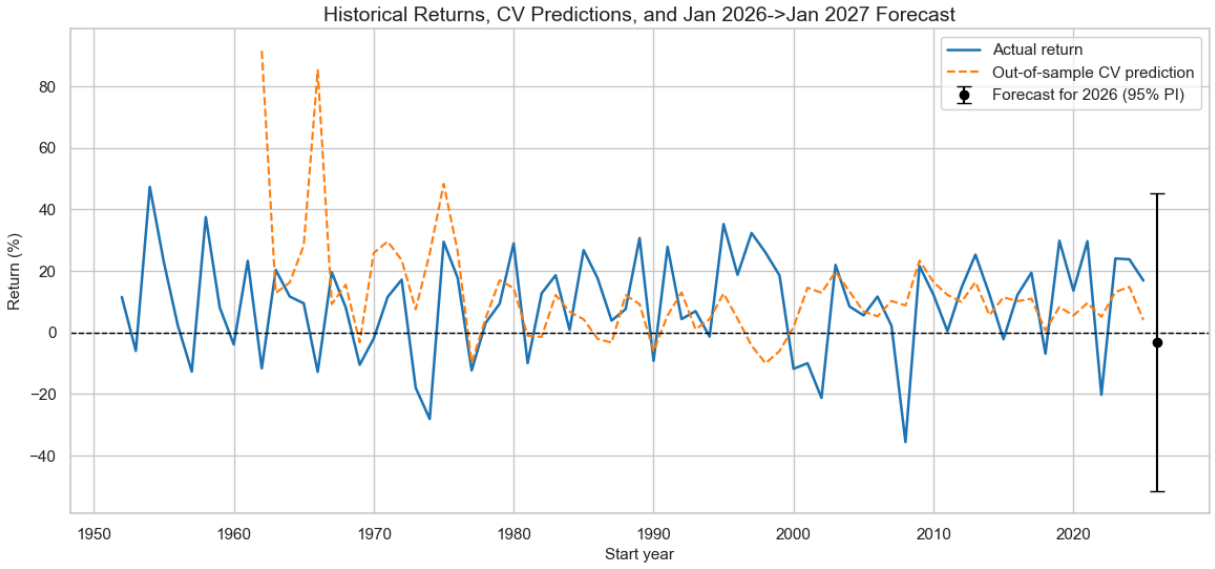
In [ ]:
```
fig, ax = plt.subplots(figsize=(14, 6))
```

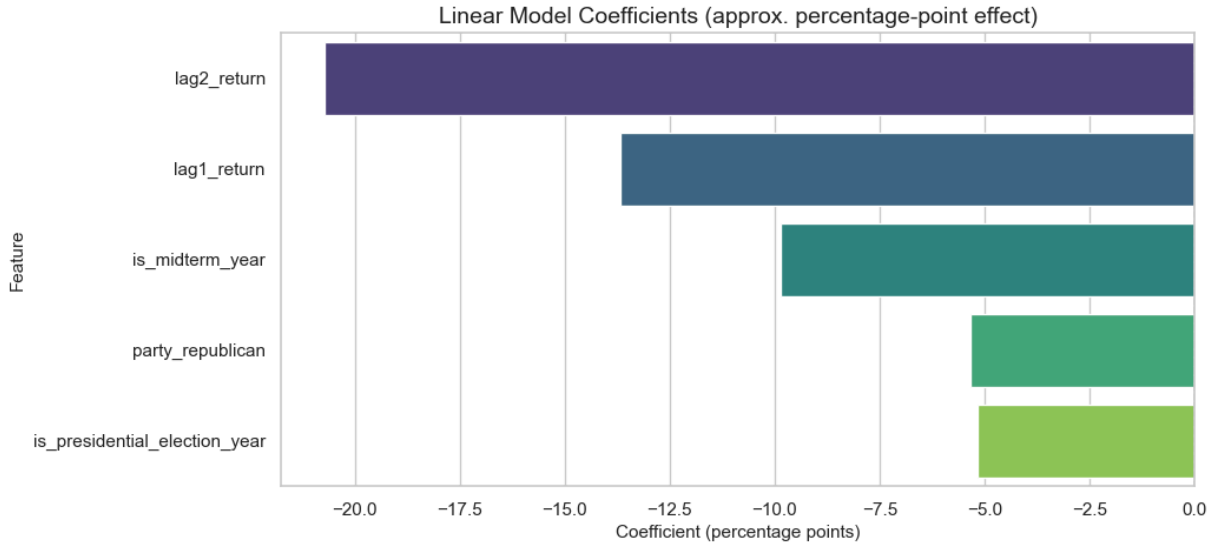
```
ax.plot(model_df.index, y * 100, color="#1f77b4", label="Actual return", lir
ax.plot(
    cv_pred.index[mask],
    cv_pred[mask] * 100,
    color="#ff7f0e",
    linestyle="--",
    label="Out-of-sample CV prediction",
)

lower_err = (forecast_return - pi_low) * 100
upper_err = (pi_high - forecast_return) * 100
ax.errorbar(
    forecast_year,
    forecast_return * 100,
    yerr=[[lower_err], [upper_err]],
    fmt="o",
    color="black",
    capsize=5,
    label="Forecast for 2026 (95% PI)",
)

ax.axhline(0, color="black", linestyle="--", linewidth=1)
ax.set_title("Historical Returns, CV Predictions, and Jan 2026->Jan 2027 For
ax.set_xlabel("Start year")
ax.set_ylabel("Return (%)")
ax.legend()
plt.show()

fig, ax = plt.subplots(figsize=(10, 5))
coef_plot = (coef_series * 100).rename("coef_pct_points")
sns.barplot(x=coef_plot.values, y=coef_plot.index, palette="viridis", ax=ax)
ax.axvline(0, color="black", linestyle="--", linewidth=1)
ax.set_title("Linear Model Coefficients (approx. percentage-point effect)")
ax.set_xlabel("Coefficient (percentage points)")
ax.set_ylabel("Feature")
plt.show()
```





In [18]:
```
text = f"""
### Critical answer for Question 3
Using a historical linear model with political-cycle indicators and lagged m
**{forecast_return*100:.2f}%**.

A rough 95% prediction interval is **[{pi_low*100:.2f}%, {pi_high*100:.2f}%]
Cross-validated error is substantial (RMSE = **{cv_rmse*100:.2f}** percentag

Interpretation: the model suggests an expected positive/negative direction b
This is best treated as a scenario estimate, not a precise forecast.
"""

# Resolve direction phrase cleanly
direction = "positive" if forecast_return >= 0 else "negative"
text = text.replace("positive/negative", direction)
display(Markdown(text))
```

Critical answer for Question 3

Using a historical linear model with political-cycle indicators and lagged market returns, the predicted S&P 500 return for **Jan 2026 to Jan 2027** is **-3.28%**.

A rough 95% prediction interval is **[-51.67%, 45.11%]**, which is wide. Cross-validated error is substantial (RMSE = **24.76** percentage points), so confidence in the point estimate should be low-to-moderate.

Interpretation: the model suggests an expected negative direction based on current inputs, but uncertainty dominates. This is best treated as a scenario estimate, not a precise forecast.

Ethical and practical considerations

- Correlation is not causation: political labels can proxy many hidden macro factors.
- Small election sample sizes limit certainty, especially for Question 2.

- Forecasting markets can be misused as certainty; communicate confidence intervals and uncertainty.
- This notebook does not use non-public information and is not financial advice.

Files created for reproducibility

- `data/sp500_daily.csv`
- `data/sp500_annual_returns_jan_to_jan.csv`
- `data/annual_returns_with_politics.csv`
- `data/us_presidential_elections_1952_2024.csv`
- `data/election_market_dataset.csv`
- `data/model_training_dataset.csv`